

Lab # 01: Data Preprocessing

Cleaning, Encoding, and Feature Scaling

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Task 1: Dataset Cleaning

1. Load the Titanic dataset (train.csv).
2. Display the first 10 rows.
3. Check for missing values in each column.
4. Fill missing values in the "Age" column with the mean age.
5. Drop rows where the "Embarked" column is missing.

```
import pandas as pd

# 1. Load the Titanic dataset (train.csv)
df = pd.read_csv('datasets/train.csv')

# 2. Display the first 10 rows
print("First 10 rows:")
display(df.head(10))
```

First 10 rows:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
5	6	0	3	
6	7	0	1	
7	8	0	3	
8	9	1	3	
9	10	1	2	

SibSp	\	Name	Sex	Age
0		Braund, Mr. Owen Harris	male	22.0
1				
1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1				
2		Heikkinen, Miss. Laina	female	26.0
0				
3		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
1				

```

4         Allen, Mr. William Henry    male  35.0
0
5         Moran, Mr. James           male   NaN
0
6         McCarthy, Mr. Timothy J    male  54.0
0
7         Palsson, Master. Gosta Leonard  male   2.0
3
8  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0
0
9         Nasser, Mrs. Nicholas (Adele Achem)  female  14.0
1

```

```

      Parch      Ticket    Fare Cabin Embarked
0         0    A/5 21171    7.2500   NaN        S
1         0      PC 17599   71.2833   C85        C
2         0  STON/O2. 3101282    7.9250   NaN        S
3         0          113803   53.1000  C123        S
4         0          373450    8.0500   NaN        S
5         0          330877    8.4583   NaN        Q
6         0          17463   51.8625   E46        S
7         1          349909   21.0750   NaN        S
8         2          347742   11.1333   NaN        S
9         0          237736   30.0708   NaN        C

```

```

# 3. Check for missing values in each column
print("\nMissing values before cleaning:")
print(df.isnull().sum())

```

```

Missing values before cleaning:
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64

```

```

# 4. Fill missing values in the "Age" column with the mean age
df['Age'] = df['Age'].fillna(df['Age'].mean())

```

```

# 5. Drop rows where the "Embarked" column is missing

```

```
df = df.dropna(subset=['Embarked'])

# Verify the changes
print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         0
dtype: int64
```

Task 2: Encoding Categorical Data

1. Convert the "Sex" column into numeric (0 = Male, 1 = Female).
2. Apply One-Hot Encoding on the "Embarked" column.

```
# 1. Convert the "Sex" column into numeric (0 = Male, 1 = Female).
# Using map to assign the numeric values based on the requirements
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

```
# 2. Apply One-Hot Encoding on the "Embarked" column.
# pd.get_dummies automatically applies one-hot encoding
# We set drop_first=True to avoid the dummy variable trap
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

```
# Display the first few rows to verify the transformations
display(df.head())
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Parch	\	Name	Sex	Age	SibSp
0			Braund, Mr. Owen Harris	0	22.0	1
0						
1			Cumings, Mrs. John Bradley (Florence Briggs Th...	1	38.0	1

0						
2			Heikkinen, Miss. Laina	1	26.0	0
0						
3			Futrelle, Mrs. Jacques Heath (Lily May Peel)	1	35.0	1
0						
4			Allen, Mr. William Henry	0	35.0	0
0						
	Ticket	Fare	Cabin	Embarked_Q	Embarked_S	
0	A/5 21171	7.2500	NaN	False	True	
1	PC 17599	71.2833	C85	False	False	
2	STON/O2. 3101282	7.9250	NaN	False	True	
3	113803	53.1000	C123	False	True	
4	373450	8.0500	NaN	False	True	

Task 3: Feature Scaling & Splitting

1. Select features: Age, Fare, Sex, Pclass.
2. Apply StandardScaler to normalize them.
3. Split data into 80% training and 20% testing.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Select features: Age, Fare, Sex, Pclass
features = ['Age', 'Fare', 'Sex', 'Pclass']
X = df[features]

# We also need to define the target variable 'y' to split the data properly
y = df['Survived']

# 2. Apply StandardScaler to normalize them
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Split data into 80% training and 20% testing
# test_size=0.2 represents the 20% testing data. random_state ensures reproducibility.
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

# Verify the shapes of the splits
print(f"Training data shape: X_train: {X_train.shape}, y_train: {y_train.shape}")
print(f"Testing data shape: X_test: {X_test.shape}, y_test: {y_test.shape}")

Training data shape: X_train: (711, 4), y_train: (711,)
Testing data shape: X_test: (178, 4), y_test: (178,)
```